COER UNIVERSITY, ROORKEE

Department of Computer Science & Engineering

Full Stack Web Development

Unit I Assignment: Full Stack Architecture & Planning

Name: Suraj Singh	**CU ID:** cu240251579
Roll No.: 59	**Subject:** Full Stack Web Development (BTCS303T)

Question 1. Write a folder structure (as a tree) for a full stack e-commerce app using React (frontend), Express (backend), and MySQL (database). In 2 lines, justify why the routes/ and models/ folders should never import each other directly.

Answer:

```
ecommerce-app/ |-- client/
# React frontend
|   |-- public/
|   |   `-- index.html
|   |-- src/
|   |   |-- assets/              # images, icons, css |   |   |-- components/
# Navbar, ProductCard, CartItem ...
|   |   |-- pages/               # Home, ProductList, Cart, Checkout, Login
|   |   |-- context/             # AuthContext, CartContext
|   |   |-- services/api.js      # all fetch/axios calls to the backend |
|   |-- App.js
|   |   `-- index.js
|   |-- .env                     # REACT_APP_API_URL
|   `-- package.json
|
|-- server/                      # Express backend
|   |-- config/
|   |   `-- db.js                # MySQL connection pool
|   |-- controllers/             # business logic
|   |   |-- authController.js
|   |   |-- productController.js
|   |   `-- orderController.js
|   |-- middleware/
|   |   |-- authMiddleware.js    # JWT verification
|   |   `-- errorHandler.js
|   |-- models/                  # SQL queries / data access
|   |   |-- User.js
|   |   |-- Product.js
|   |   `-- Order.js
|   |-- routes/                  # URL -> controller mapping
|   |   |-- authRoutes.js
|   |   |-- productRoutes.js
|   |   `-- orderRoutes.js
|   |-- utils/ |   |-- .env                    # PORT,
DB_URL, JWT_SECRET
|   |-- server.js                # entry point
|   `-- package.json
|
|-- database/
|   |-- schema.sql               # CREATE TABLE statements
|   `-- seed.sql                 # sample data
|
|-- .gitignore
|-- package.json                 # root scripts (concurrently)
`-- README.md
```

Justification (2 lines):

1. routes/ handle HTTP (req/res) and models/ handle SQL/data, so direct imports tightly couple the twolayers.

2. This causes circular dependencies and hard-to-test code; a controller should sit between them (routes ->controllers -> models).

Question 2. This React component fetches data using a hardcoded localhost URL. Explain in 2 lines why this call will fail once deployed to production, and rewrite the fetch line using an environment variable.

```
function ProductList() {   useEffect(() => {     fetch("http://localhost:5000/api/products")
.then(res => res.json())
      .then(setProducts);
}, []);   ...
}
```

Answer:

Explanation (2 lines):

1. After deployment, *localhost* in the browser points to the user's own device, where no API server is running.

2. The URL is also hardcoded, so it must be edited by hand for every environment (dev / staging / production).

Rewritten fetch line:

```
fetch(`${process.env.REACT_APP_API_URL}/api/products`)
```

Supporting .env files (inside client/):

```
# client/.env  (development)
REACT_APP_API_URL=http://localhost:5000


# client/.env.production  (used by npm run build)
REACT_APP_API_URL=https://api.myshop.com
```

Note: in Create React App the variable must start with `REACT_APP_`; in Vite use `import.meta.env.VITE_API_URL` instead.

Question 3. Write a .env file and the corresponding dotenv-loading code for a Node.js/Express backend that needs PORT, DB_URL, and JWT_SECRET. In 1 line, explain why .env should never be committed to Git.

Answer:

.env file (server/.env):

```
PORT=5000
DB_URL=mysql://root:mypassword@localhost:3306/ecommerce_db
JWT_SECRET=s3cr3t_k3y_change_this_to_a_long_random_string
```

dotenv-loading code (server.js): `npm install dotenv`

```
require('dotenv').config();          // must be called before using
process.env const express = require('express'); const app = express();  const
PORT       = process.env.PORT || 5000; const DB_URL     = process.env.DB_URL;
const JWT_SECRET = process.env.JWT_SECRET;  if (!DB_URL || !JWT_SECRET) {
console.error('Missing required environment variables in .env');
```

```
process.exit(1); }  app.listen(PORT, () => console.log(`Server running on
port ${PORT}`));
```

Why never commit .env (1 line): It contains secrets (DB password, JWT secret) that would stay visible in Git history to anyone with repo access; add `.env` to `.gitignore`.

Question 4. A React frontend calling an Express API gets a CORS error in the browser console on every request. Write the exact Express middleware/package code that fixes it, and explain in 1 line why the browser blocks the request in the first place.

Answer:

Fix: `npm install cors`

```
const express = require('express');
const cors = require('cors'); const
app = express();
app.use(cors({   // in production, replace with your deployed
frontend URL   origin: 'http://localhost:3000',
// React app URL   methods: ['GET', 'POST', 'PUT',
'DELETE'],   allowedHeaders: ['Content-Type', 'Authorization'],
credentials: true }));  app.use(express.json());
// ... routes go below (cors must be registered BEFORE the routes)
```

Quick version for development only (allows every origin): `app.use(cors());`

Why the browser blocks it (1 line): The Same-Origin Policy blocks the response because the React app (localhost:3000) and the API (localhost:5000) have different origins and the server sent no *Access-ControlAllow-Origin* header.

Question 5. Design a REST API contract (as a table: Endpoint | Method | Request Body | Response) for a "Book Library" system with 4 endpoints covering create, read, update, and delete.

Answer:

Endpoint	Method	Request Body	Response
`/api/books`	**POST**	{ "title": "Clean Code", "author": "Robert C. Martin", "isbn": "9780132350884", "year": 2008 }	201 Created { "id": 1, "title": "Clean Code", "author": "Robert C. Martin", "isbn": "9780132350884", "year": 2008 }
`/api/books`	**GET**	None	200 OK [{ "id": 1, "title": "Clean Code", ... }, { "id": 2, ... }]
`/api/books/:id`	**PUT**	{ "title": "Clean Code (2nd Ed.)", "author": "Robert C. Martin", "isbn": "9780132350884", "year": 2009 }	200 OK { "id": 1, "title": "Clean Code (2nd Ed.)", ... } 404 Not Found if id does not exist

/api/books/:id	**DELETE**	None	200 OK { "message": "Book deleted successfully" } 404 Not Found if id does not exist

Question 6. This Express code creates a new database connection inside every route handler. Rewrite it so the connection is created once and reused across routes, and explain in 2 lines why the original version is a performance problem.

```
app.get('/products', (req, res) => {   const db =
mysql.createConnection({ host: 'localhost', user: 'root' });   db.query('SELECT
* FROM products', (err, rows) => res.json(rows));
});
```

Answer:

Rewritten code (`npm install mysql2` **- same API, works with MySQL 8):**

```
// config/db.js   -> runs ONCE when the app starts
const  mysql  =  require('mysql2');    const  db  =
mysql.createPool({     host:  'localhost',     user:
'root',        password:   process.env.DB_PASSWORD,
database: 'shop',   connectionLimit: 10          //
up to 10 reusable connections });  module.exports =
db;
 // server.js / routes -> reuse the same db object everywhere const
db = require('./config/db');  app.get('/products', (req, res) => {
db.query('SELECT * FROM products', (err, rows) => {     if (err)
return res.status(500).json({ error: 'Database error' });
res.json(rows);   });
});
```

Why the original is a performance problem (2 lines):

1. Every request opens a brand-new TCP connection plus MySQL handshake/authentication, adding latencyto each call.

2. The connection is never closed, so they pile up until MySQL hits *max_connections* and starts refusing requests.

Question 7. Write the "scripts" section of package.json for a full stack project where npm run dev should start both the backend (nodemon server.js) and the frontend (npm start inside a client folder) together, using the concurrently package.

Answer:

Install: `npm install --save-dev concurrently nodemon` **package.json**

(project root):

```
"scripts": {
  "server": "nodemon server.js",
```

```
  "client": "npm start --prefix client",
  "dev": "concurrently \"npm run server\" \"npm run client\"",
  "start": "node server.js"
}
```

Running `npm run dev` now launches the Express backend (with auto-restart via nodemon) and the React app in the same terminal. `start` is used in production.

Question 8. A teammate's code hardcodes an API key directly inside a React component. Rewrite the code using an environment variable instead, and explain the security risk in 2 lines.

```
const response = await fetch(
`https://api.weather.com/data?key=sk_live_9f8a7b6c5d4e`
);
```

Answer:

Rewritten code:

```
// client/.env   (add this file to .gitignore)
REACT_APP_WEATHER_API_KEY=sk_live_9f8a7b6c5d4e
 // React component const
response = await fetch(
`https://api.weather.com/data?key=${process.env.REACT_APP_WEATHER_API_KEY}`
);
```

Security risk (2 lines):

1. React code is shipped to the browser, so anyone can read a hardcoded key via DevTools/View Source(and it also gets committed to Git).

2. A leaked *sk_live* key lets attackers make unlimited paid API calls in your name, exhausting your quota or billing.

Best practice: React env variables are still bundled into the JavaScript at build time, so a truly secret key should be kept on the Express server and the frontend should call your own endpoint:

```
// server.js (key stays in server/.env as WEATHER_API_KEY, never sent to the browser)
app.get('/api/weather', async (req, res) => {   const r = await
fetch(`https://api.weather.com/data?key=${process.env.WEATHER_API_KEY}`);
res.json(await r.json()); });
 // React const response = await
fetch(`${process.env.REACT_APP_API_URL}/api/weather`);
```

Question 9. Write pseudocode (as commented steps) tracing the full request-response cycle when a user submits a login form in a full stack app - from the button click to the database query to the UI update.

Answer:

```
// ---------------- FRONTEND (React) ---------------// STEP 1: User types username
and password; onChange updates React state.
// STEP 2: User clicks the "Login" button -> onSubmit handler runs.
//         e.preventDefault() stops the page from reloading; basic validation
//         (fields not empty) is done.
// STEP 3: React sends the request:
//         fetch(API_URL + '/api/auth/login', { method: 'POST',
//           headers: {'Content-Type': 'application/json'}, //
body: JSON.stringify({ username, password }) })
// STEP 4: Browser sends the HTTP request (a CORS preflight OPTIONS request
//         goes first if the API is on a different origin).


// ---------------- BACKEND (Express) ---------------// STEP 5: Express receives
the request and passes it through middleware:
//         cors() -> express.json() (parses body into req.body).
// STEP 6: Router matches POST /api/auth/login and calls authController.login.
// STEP 7: Controller reads { username, password } from req.body and validates.
// STEP 8: Model runs a parameterized SQL query through the shared MySQL pool: //
SELECT id, username, password_hash FROM users WHERE username = ?


// ---------------- DATABASE (MySQL) ----------------
// STEP 9: MySQL finds the matching row (or returns an empty result) and sends //
it back to the Express server.


// ---------------- BACKEND (Express) ----------------
// STEP 10: If no user is found        -> res.status(401).json({ error: 'Invalid credentials'
}) // STEP 11: bcrypt.compare(password,
password_hash)
//          If it does not match       -> res.status(401).json({ error: 'Invalid credentials'
})
// STEP 12: If it matches, create token = jwt.sign({ id }, JWT_SECRET, { expiresIn: '1h' }) //
STEP 13: Send response: res.status(200).json({ token, user: { id, username } })


// ---------------- FRONTEND (React) ---------------// STEP 14:
Browser delivers the response; React runs res.json().
// STEP 15: If response is OK  -> store token (memory / httpOnly cookie / localStorage), //
update AuthContext state, redirect to /dashboard.
//          If response failed -> set error state and show "Invalid username or password".
// STEP 16: UI re-renders with the new state (logged-in view or error message).
//          Future requests send the header: Authorization: Bearer <token>
```

Question 10. This Express server crashes on the very first POST request. Identify the missing line that causes req.body to be undefined, and provide the corrected code.

```
const express = require('express'); const
app = express();  app.post('/login', (req,
res) => {   const { username, password } =
req.body;   res.send(`Welcome
${username}`); });  app.listen(3000);
```

Answer:

Missing line: `app.use(express.json());` - without a body-parsing middleware Express leaves `req.body` as *undefined*, so destructuring it throws a TypeError.

Corrected code:

```
const express =
require('express'); const app = express();
app.use(express.json());   // <-- MISSING LINE: parses JSON request bodies
// optional: also parse HTML form submissions app.use(express.urlencoded({
extended: true }));  app.post('/login',
(req, res) => {   const { username, password } = req.body;   res.send(`Welcome
${username}`); });  app.listen(3000);
```